

CARONAUNI · DOCUMENTAÇÃO TÉCNICA

Plataforma de Caronas Universitárias

Documentação Técnica Evolutiva — v2.0

Unoesc Chapecó

Engenharia de Software

Semestre 2026/1

Protótipo Funcional

INFORMAÇÕES DO DOCUMENTO

PROJETO

CaronaUni

VERSÃO

2.0 — Evolutiva

INSTITUIÇÃO

**Unoesc — Campus
Chapecó**

CURSO

Engenharia de Software

PERÍODO

2026/1

STATUS

Protótipo Publicado

Sumário

CaronaUni

Universidade do Oeste de Santa Catarina ·
UnoescChapecó — SC ·
2026

1.	Descrição do Sistema	3
2.	Escopo do Sistema	4
3.	Requisitos do Sistema	5
	3.1 Requisitos Funcionais	5
	3.2 Requisitos Não Funcionais	5
4.	Arquitetura do Sistema	6
5.	Protótipo Navegável	7
6.	Modelo Entidade-Relacionamento (ER)	8
	6.1 Diagrama de Relacionamentos	8
	6.2 Estrutura das Entidades	8
7.	Matriz de Rastreabilidade	9
8.	Plano de Execução	10

SEÇÃO 1

Descrição do Sistema

O **CaronaUni** é uma plataforma web de caronas universitárias desenvolvida para conectar, de forma simples e segura, estudantes da **Unoesc (Universidade do Oeste de Santa Catarina)** que oferecem vagas em seus veículos com estudantes que precisam de deslocamento até o campus.

A plataforma resolve problemas recorrentes na comunidade acadêmica: a dificuldade de encontrar caronas confiáveis, a ausência de um processo organizado para solicitar e confirmar vagas, e a falta de histórico e avaliação para aumentar a confiança entre os usuários.

O sistema é desenvolvido como um protótipo funcional de alta fidelidade em HTML, CSS e JavaScript puro, com mapa interativo via **Leaflet.js + OpenStreetMap**, hospedado no GitHub Pages para acesso público sem necessidade de infraestrutura de servidor.

Propósito

- Facilitar o deslocamento diário da comunidade acadêmica
- Reduzir custos individuais de transporte
- Diminuir o número de veículos em circulação
- Promover sustentabilidade no campus
- Aumentar a confiança entre usuários via avaliações

Fluxo Principal

1. Usuário faz login via SSO Unoesc (mock)
2. Motorista cria carona com rota e horário
3. Passageiro busca e solicita uma vaga
4. Motorista aceita ou recusa a solicitação
5. Carona é realizada e ambos se avaliam
6. Reputação de ambos é atualizada

Público-alvo

Estudantes regularmente matriculados na Unoesc — tanto motoristas que desejam compartilhar viagens e reduzir seus custos, quanto passageiros que buscam deslocamento confiável até o campus. O acesso é exclusivo a membros da comunidade acadêmica, garantido via autenticação institucional.

Contexto do projeto: Esta é uma entrega evolutiva do semestre 2026/1, dando continuidade ao levantamento e modelagem realizados no semestre anterior. O foco desta fase é o protótipo funcional navegável, o modelo ER e a documentação técnica consolidada.

SEÇÃO 2

Escopo do Sistema

✓ Incluso — O que o sistema contempla

- Login e identificação de usuário (SSO mock)
- Cadastro de carona com rota, horário e vagas
- Mapa interativo para definição de trajeto
- Busca de caronas disponíveis com filtros
- Solicitação de vaga pelo passageiro
- Aceite ou recusa da solicitação pelo motorista
- Registro de avaliação pós-carona
- Histórico de caronas realizadas
- Gerenciamento de perfil e veículos
- Sistema de notificações (simulado)
- Caronas recorrentes por dia da semana

✗ Fora de Escopo

- Pagamento integrado
- Aplicativo mobile nativo completo
- Otimização avançada de rotas (Waze/Maps)
- Integração real com SSO da Unoesc
- Chat em tempo real (WebSocket)
- Backend e banco de dados reais
- Notificações push nativas
- Rastreamento GPS em tempo real

📌 **Modalidade de execução:** Continuação evolutiva do projeto do semestre anterior. Foco em documentação evolutiva, modelo ER e protótipo funcional navegável. Modelo de processo: **Ágil Incremental (iterativo)** com entregas por iteração.

Restrições Técnicas

Restrição	Descrição	Impacto
Sem backend real	Toda a lógica é simulada no frontend com dados estáticos	Dados não persistem entre sessões

Restrição	Descrição	Impacto
SSO mock	Autenticação simulada — qualquer credencial é aceita	Sem validação real de matrícula
Mapa estático	OpenStreetMap via Leaflet, sem roteamento real	Rotas são linhas retas entre pontos
Hospedagem estática	GitHub Pages — apenas arquivos HTML/CSS/JS	Sem processamento server-side

SEÇÃO 3

Requisitos do Sistema

3.1 Requisitos Funcionais

ID	Descrição	Prioridade	Status
RF-P1	Login de usuário via SSO Unoesc (real ou mock) com identificação e autenticação	Alta	✓ Implementado
RF-P2	Criar carona com origem, destino, data/hora, número de vagas e preferências do motorista	Alta	✓ Implementado
RF-P3	Buscar caronas disponíveis e visualizar detalhes da carona e do motorista	Alta	✓ Implementado
RF-P4	Solicitar vaga em carona com mensagem opcional para o motorista	Alta	✓ Implementado
RF-P5	Aceitar ou recusar solicitação com atualização automática de vagas e notificação	Alta	✓ Implementado
RF-P6	Avaliar usuário após carona concluída com nota, critérios detalhados e comentário	Média	✓ Implementado
RF-P7	Visualizar histórico de caronas (realizadas, futuras, pendentes de avaliação)	Média	✓ Implementado
RF-P8	Gerenciar perfil do usuário, veículos cadastrados e preferências de carona	Média	✓ Implementado

3.2 Requisitos Não Funcionais

ID	Descrição	Categoria
RNF-P1	Arquitetura em camadas com separação clara entre Frontend, API REST e Banco de Dados	Arquitetura
RNF-P2	Comunicação segura via HTTPS e proteção de dados pessoais dos usuários	Segurança

ID	Descrição	Categoria
RNF-P3	Validação para evitar inconsistência de vagas em situações de acesso concorrente	Consistência
RNF-P4	Interface responsiva com feedback visual claro de sucesso e erro para o usuário	Usabilidade

SEÇÃO 4

Arquitetura do Sistema

O sistema segue uma arquitetura em três camadas com separação clara de responsabilidades entre Interface Web, Backend e Banco de Dados. No protótipo atual, apenas a camada de Interface está implementada; as demais são modeladas para a fase de desenvolvimento real.

Frontend — Interface Web

Protótipo em HTML5, CSS3 e JavaScript puro. Responsivo e totalmente navegável. Mapa interativo via Leaflet.js com tiles OpenStreetMap. Geocodificação reversa via Nominatim API.

HTML / CSS / JS

Leaflet.js

OpenStreetMap

Backend — API REST

Camada de serviços responsável pela lógica de negócio: autenticação via JWT, gerenciamento de caronas, controle de solicitações, avaliações e mensagens entre usuários.

Node.js / Python

REST API

JWT Auth

Banco de Dados

Armazenamento relacional das entidades principais do domínio: Usuário, Carona, Solicitação, Avaliação e Mensagem, com integridade referencial via chaves estrangeiras.

PostgreSQL / MySQL

Modelo Relacional

SSO Institucional

Integração com o sistema de autenticação da Unoesc para validação de matrícula ativa. No protótipo atual, o SSO é simulado — qualquer credencial @unoesc.edu.br é aceita.

OAuth 2.0 / SAML

Mock no protótipo

Fluxo de Dados

Requisições comuns:

Interface Web → HTTPS → API REST → Serviços (Carona / Usuário / Mensagem) → Banco de Dados

Autenticação:

Interface Web → SSO Unoesc → Token JWT → API REST (header Authorization)

Tecnologias Utilizadas no Protótipo

Tecnologia	Finalidade	Tipo
HTML5 / CSS3 / JS	Estrutura, estilo e lógica de interação das telas	Frontend
Leaflet.js	Mapa interativo com marcadores, polilinha de rota e eventos de clique	Frontend
OpenStreetMap	Tiles de mapa gratuitos, sem necessidade de chave de API	Serviço externo
Nominatim API	Geocodificação reversa: clique no mapa → endereço textual	Serviço externo
GitHub Pages	Hospedagem estática gratuita para o protótipo	Infraestrutura

SEÇÃO 5

Protótipo Navegável

O protótipo foi desenvolvido em HTML/CSS/JavaScript puro, sem dependências de build ou bundler. Está hospedado no GitHub Pages e pode ser acessado publicamente pelo link abaixo, sem necessidade de instalação ou conta.

 **Link do Protótipo:**

<https://viniciusoliveira-ai.github.io/Site-Carona/>

Telas Implementadas



Login

Autenticação via SSO Unoesc
+ formulário manual com e-mail
e senha institucional



Dashboard

Estatísticas pessoais,
notificações recentes, próximas
caronas e atalhos rápidos



Buscar Carona

Mapa interativo, lista de
caronas disponíveis com filtros
por horário e distância



Oferecer Carona

Criação com mapa (clique =
origem, duplo clique = destino),
vagas e recorrência



Detalhes

Informações da carona, perfil e
avaliações do motorista, mapa
de rota



Solicitações

Gerenciamento das
solicitações recebidas pelo
motorista: aceitar ou recusar



Avaliação

Sistema de estrelas com
critérios detalhados, tags
rápidas e comentário



Histórico

Registro de caronas passadas,
futuras e pendentes de
avaliação, com abas



Perfil

Dados pessoais, veículos
cadastrados, reputação
detalhada e preferências

Critério de pronto atendido: Fluxo ponta a ponta funcionando — login → buscar carona → solicitar vaga → aceitar solicitação → avaliar usuário. Mapa navegável com definição de origem e destino por clique. Protótipo hospedado e acessível via link público.

Fluxo de Navegação Principal

O usuário inicia em `index.html` (Login) → é redirecionado para `dashboard.html` → pode buscar em `buscar.html`, ver detalhes em `detalhes.html` e solicitar uma vaga. O motorista gerencia

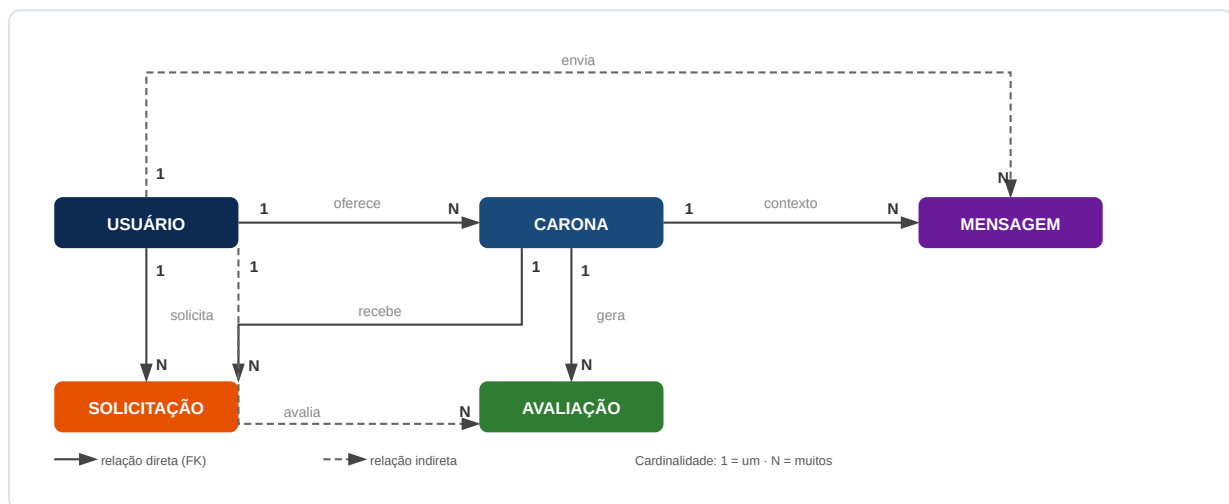
solicitações em `solicitacoes.html`. Após a carona, o passageiro avalia em `avaliacao.html`. Todo o histórico fica em `historico.html` e o perfil em `perfil.html`.

SEÇÃO 6

Modelo Entidade-Relacionamento

O modelo ER representa a estrutura de dados do sistema, derivada do Diagrama de Classes e dos requisitos funcionais levantados. O banco é composto por cinco entidades principais com relacionamentos do tipo 1:N.

6.1 Diagrama de Relacionamentos



6.2 Estrutura das Entidades

USUÁRIO	
PK id	INT AUTO
nome	VARCHAR(120)
email	VARCHAR(150) UNIQUE
senha_hash	VARCHAR(255)
reputação	DECIMAL(3,2)
matricula	VARCHAR(20) UNIQUE
created_at	TIMESTAMP

CARONA	
PK id	INT AUTO
FK motorista_id	→ usuario.id
origem	VARCHAR(255)
destino	VARCHAR(255)
data_hora	DATETIME
vagas_total	TINYINT
vagas_disp	TINYINT
status	ENUM(ativa, concluída, cancelada)
created_at	TIMESTAMP

MENSAGEM	
PK id	INT AUTO
FK remetente_id	→ usuario.id
FK destinat_id	→ usuario.id
FK carona_id	→ carona.id
conteúdo	TEXT
data_hora	DATETIME

SOLICITAÇÃO	
PK id	INT AUTO
FK carona_id	→ carona.id
FK passageiro_id	→ usuario.id
status	ENUM(pendente, aceito, recusado)
mensagem	TEXT NULLABLE
created_at	TIMESTAMP

AVALIAÇÃO	
PK id	INT AUTO
FK avaliador_id	→ usuario.id
FK avaliado_id	→ usuario.id
FK carona_id	→ carona.id
nota	TINYINT (1 a 5)
comentario	TEXT NULLABLE
created_at	TIMESTAMP

PK = Chave Primária **FK** = Chave Estrangeira

AVALIAÇÃO possui 2 FKs para USUÁRIO (avaliador e avaliado)

MENSAGEM possui 2 FKs para USUÁRIO (remetente e destinatário)

SEÇÃO 7

Matriz de Rastreabilidade

A matriz abaixo relaciona cada requisito funcional aos artefatos de modelagem correspondentes (caso de uso, diagrama de sequência, diagrama de atividade) e à tela do protótipo onde foi implementado.

Requisito	Caso de Uso	Diagrama de Sequência	Diagrama de Atividade	Tela no Protótipo
RF-P1 – Login	Fazer Login	Autenticação	—	index.html
RF-P2 – Criar carona	Criar Carona	—	Criar Carona	criar.html
RF-P3 – Buscar carona	Buscar Carona	—	Buscar e Solicitar	buscar.html / detalhes.html
RF-P4 – Solicitar vaga	Solicitar Vaga	Solicitar Carona	Buscar e Solicitar	detalhes.html
RF-P5 – Aceitar/Recusar	Aceitar Solicitação	Solicitar Carona	Processar Solicitação	solicitacoes.html
RF-P6 – Avaliar	Avaliar Usuário	Avaliar Carona	—	avaliacao.html
RF-P7 – Histórico	—	—	—	historico.html
RF-P8 – Perfil	Editar Perfil	—	—	perfil.html

Nota: Os diagramas de Caso de Uso, Sequência e Atividade foram desenvolvidos no semestre anterior e compõem o portfólio de modelagem do projeto. Esta documentação faz referência a eles sem reproduzi-los integralmente para manter o foco no escopo evolutivo desta entrega.

SEÇÃO 8

Plano de Execução

O projeto segue um modelo ágil incremental, dividido em iterações com entregas parciais e funcionais. As iterações 1 a 5 já foram concluídas nesta fase. A iteração 6 (Backend) está planejada para o próximo semestre.

Iteração 1 — Levantamento e Modelagem

Revisão dos requisitos do semestre anterior. Atualização dos diagramas de caso de uso, classes, sequência e atividades. Definição do escopo da entrega evolutiva.

Iteração 2 — Protótipo: Login + Dashboard + Buscar

Desenvolvimento das telas de login, dashboard principal e busca de caronas com mapa interativo Leaflet.js + OpenStreetMap e geocodificação via Nominatim.

Iteração 3 — Protótipo: Criar + Solicitar + Aceitar

Tela de criação de carona com mapa interativo (clique para origem e destino), fluxo de solicitação de vaga pelo passageiro e gerenciamento de solicitações pelo motorista.

Iteração 4 — Protótipo: Avaliação + Histórico + Perfil

Tela de avaliação pós-carona com critérios detalhados e tags rápidas, histórico de caronas com abas e filtros, e tela de perfil com preferências e veículos.

Iteração 5 — Modelo ER e Documentação Técnica

Elaboração do modelo entidade-relacionamento compatível com o fluxo principal do sistema, documentação técnica evolutiva consolidada e publicação do protótipo via GitHub Pages.

Próximo — Iteração 6: Backend (API REST + Banco de Dados)

Implementação real da API com Node.js ou Python (FastAPI/Django), banco de dados PostgreSQL, autenticação JWT e integração com SSO da Unoesc. Substituição dos dados mockados por chamadas reais à API.

Resumo das Entregas

Iteração	Entregável	Status
1	Diagramas atualizados (Caso de Uso, Classes, Sequência, Atividade)	✓ Concluído
2	Telas: Login, Dashboard, Buscar Carona (com mapa)	✓ Concluído
3	Telas: Oferecer Carona, Detalhes, Solicitações	✓ Concluído

Iteração	Entregável	Status
4	Telas: Avaliação, Histórico, Perfil	✓ Concluído
5	Modelo ER, Documentação técnica, Deploy GitHub Pages	✓ Concluído
6	Backend real (API REST + PostgreSQL + SSO)	Planejado